

Análisis Arquitectónico: Implementación de Microservicios en Producción

Preparatorio para Informe Académico TA-IND-01

Carlos Carpio Mendoza

Universidad Técnica Estatal de Quevedo

4 de junio de 2026

DESCRIPCIÓN DEL CASO: NETFLIX

A finales de 2008, Netflix sufrió una grave corrupción de base de datos que paralizó su servicio de envío de DVD durante tres días. Este incidente evidenció las limitaciones de su arquitectura monolítica tradicional ante un crecimiento exponencial de usuarios. Para resolverlo, la empresa inició una transición progresiva hacia una arquitectura basada en **Microservicios** desplegada en la nube de AWS, un proceso que culminó en 2016. Hoy en día, la plataforma gestiona miles de microservicios independientes que interactúan mediante llamadas REST y pasarelas de enrutamiento (API Gateway) para servir streaming de video a cientos de millones de usuarios simultáneos a nivel global.

JUSTIFICACIÓN DEL ESTILO ARQUITECTÓNICO

La elección de los microservicios se fundamentó en tres pilares críticos de los sistemas distribuidos:

- **Escalabilidad Horizontal Dinámica:** Un monolito obligaba a escalar toda la aplicación de forma vertical. Los microservicios permiten escalar de manera independiente únicamente los componentes con alta carga de cómputo (como el motor de recomendaciones o el procesamiento de video).
- **Aislamiento de Fallos (Fault Isolation):** En un entorno monolítico, un fallo de memoria en un módulo secundario puede derribar todo el sistema. En la arquitectura distribuida, si el servicio de búsqueda falla, el usuario aún puede reproducir contenido de su lista principal.
- **Velocidad de Despliegue e Innovación:** Permite a equipos de desarrollo autónomos (Two-Pizza Teams) actualizar, probar y desplegar servicios específicos múltiples veces al día sin coordinar un despliegue masivo y riesgoso.

BENEFICIOS OBTENIDOS Y DESAFÍOS ENCONTRADOS

La adopción de esta arquitectura transformó la resiliencia de la empresa, dando origen a la práctica de *Chaos Engineering* mediante herramientas como el *Simian Army* (Chaos Monkey), que apaga servicios en producción para garantizar la tolerancia a fallos. Además, optimizaron el tráfico de red centralizando la autenticación y el enrutamiento a través de un **API Gateway** (Zuul).

Sin embargo, los desafíos técnicos fueron complejos. El principal reto fue mantener la **consistencia de los datos** sin depender de transacciones distribuidas pesadas (ACID), lo que requirió adoptar el principio de consistencia eventual y patrones como *CQRS/Saga*. Asimismo, la latencia de red incrementó significativamente debido a las múltiples llamadas inter-servicio, obligando a implementar robustas estrategias de almacenamiento en caché distribuido (como Memcached y Redis) y el uso de interruptores de circuito (*Circuit Breakers* con Hystrix).

APLICACIÓN AL PROYECTO PROPIO

Este caso de estudio proporciona directrices arquitectónicas directas para el diseño de nuestro proyecto de software distribuido. En primer lugar, la separación de dominios implementada por Netflix demuestra la importancia de desacoplar los módulos principales (como la autenticación de usuarios, la gestión de inventario y el procesamiento de transacciones o ventas) en servicios autónomos que utilicen sus propias bases de datos.

En segundo lugar, la integración de un ****API Gateway**** es fundamental para actuar como punto único de entrada, abstrayendo la complejidad de la infraestructura interna para los clientes móviles o web, gestionando la seguridad mediante tokens JWT y distribuyendo las peticiones de forma eficiente. Finalmente, la contenerización de estos componentes mediante Docker y su orquestación en Kubernetes permitirá emular la elasticidad y el aislamiento de fallos observados en el caso de producción analizado.

REFERENCIAS ACADÉMICAS

Referencias

- [1] P. Di Francesco, P. Lago y I. Malavolta, “Microservices architectural migration: A systematic mapping study,” *Journal of Systems and Software*, vol. 146, pp. 100–120, dic. 2018. Disponible en: <https://doi.org/10.1016/j.jss.2018.09.016>
- [2] N. Alshuqayran, N. Ali y R. Evans, “A Systematic Mapping Study in Microservices Architecture,” en *IEEE International Conference on Service-Oriented Computing and Applications (SOCA)*, Macau, China, 2016, pp. 44–51. Disponible en: <https://doi.org/10.1109/SOCA.2016.15>